

JOB MARKET TRACKER

Egypt's End-to-End Data Engineering & Analytics Platform

Full Project Implementation Plan

Data Engineering · Analytics · Web Application



■ Project Overview

What is Job Market Tracker?

Job Market Tracker is a graduation data engineering project that builds a fully automated pipeline to collect, clean, transform, and analyze job market data from publicly available Egyptian sources (Wuzzuf, Forasna, LinkedIn, and others). The system continuously ingests job postings, processes them into structured datasets stored in PostgreSQL, and exposes rich analytics through a public-facing web application.

The Two-Phase Vision

■ Phase A — Data Engineering	■ Phase B — Software Engineering
Python · Apache Airflow · dbt · PostgreSQL Automated pipeline: ingest → clean → model → analyze	FastAPI backend · React/Next.js frontend · Cloud deployment Web app serving salary insights, demand trends & company data

End-to-End Pipeline Flow

1	Data Sources (Wuzzuf / Forasna / LinkedIn APIs & RSS Feeds) ▼
2	Ingestion — Python scripts fetch raw JSON/XML job postings ▼
3	Cleaning — Standardize titles, salaries, locations, skills ▼
4	Modeling — dbt transforms data into star-schema fact & dimension tables ▼
5	Orchestration — Airflow DAGs schedule & automate every run ▼
6	Storage — PostgreSQL holds the clean, structured dataset ▼
7	Analytics — SQL models answer market questions (salary, demand, trends) ▼
8	Backend API — FastAPI exposes analytics to the frontend ▼
9	Web App — Users explore Egypt's job market via dashboards & search

Analytics & Insights Delivered

■ Salary Intelligence	Average & range salaries by title, seniority, city, industry
■ In-Demand Jobs	Which roles are being posted the most right now
■ Oversupplied Roles	Jobs with high competition (market saturation)
■ Company Insights	Top hiring companies, sector dominance, activity trends
■ Geographic Distribution	Job density map across Egyptian cities
■ Skills Trends	Most requested skills/tools per role and industry
■ Hiring Patterns	Seasonal trends, growth sectors, declining roles over time
■ Work Mode Breakdown	Remote vs on-site vs hybrid distribution

■ ■ Data Engineering Phases (Phases 1 – 5)

01

Environment Setup & Project Structure

Data Engineering



Goal: *Get everything installed and organized before writing a single line of pipeline code.*

Tasks:

- Set up Python virtual environment and install all required libraries
- Install and configure PostgreSQL locally
- Install and initialize Apache Airflow (LocalExecutor setup)
- Install dbt and connect it to PostgreSQL
- Design and create the project folder structure
- Set up Git repository and GitHub for version control
- Create a .env file for credentials and environment configuration

■ Estimated Effort: **2–3 days**

OUTPUT

A clean, working local development environment ready for pipeline development.

02

Data Source Research & Ingestion Scripts

Data Engineering



Goal: *Identify where the data comes from and write scripts to reliably collect it.*

Tasks:

- Research available Egyptian job sources: Wuzzuf, Forasna, LinkedIn, etc.
- Identify which sources offer APIs, RSS feeds, or require web scraping
- Write Python ingestion scripts for each data source
- Store raw data as-is (JSON/XML) in a raw layer in PostgreSQL or flat files
- Handle pagination, rate limiting, and HTTP request errors
- Implement incremental loading logic — only fetch new or updated postings each run
- Log each ingestion run with source, record count, and timestamp

■ Estimated Effort: **1.5–2 weeks**

OUTPUT

Raw job postings continuously collected and stored from at least 2–3 Egyptian job sources.

03

Data Cleaning & Standardization

Data Engineering



Goal: Turn messy, inconsistent raw data into clean, usable records.

Tasks:

- Parse and normalize job titles to consistent categories
- Clean and standardize salary fields (extract min/max, currency, period)
- Normalize location fields — map to Egyptian cities and governorates
- Extract skills and technologies from job descriptions (NLP / keyword matching)
- Handle nulls, duplicates, and inconsistent date/text formats
- Classify job types: full-time, part-time, remote, hybrid, contract
- Apply data quality rules and flag records that fail validation

■ Estimated Effort: 1–1.5 weeks

OUTPUT

A clean, standardized dataset ready for relational modeling and transformation.

04

Data Modeling & PostgreSQL Schema Design

Data Engineering



Goal: Design a solid relational schema that efficiently supports all analytics queries.

Tasks:

- Design a star schema with fact and dimension tables
- Create fact_job_postings table (core metrics per posting)
- Create dimension tables: dim_company, dim_location, dim_skills, dim_job_category
- Write SQL DDL scripts to create all tables with correct types and constraints
- Define primary keys, foreign keys, and performance indexes
- Plan for slowly changing dimensions (e.g. company info updates over time)
- Seed lookup/reference tables with Egyptian cities and job categories

■ Estimated Effort: 4–5 days

OUTPUT

A well-structured PostgreSQL star schema ready to receive and serve clean data.

05

dbt Transformation Layer

Data Engineering



Goal: Use dbt to build reliable, version-controlled, tested transformation models.

Tasks:

- Set up dbt project and connect to PostgreSQL
- Build staging models — map raw cleaned data into the schema
- Build intermediate models — join, enrich, and aggregate across tables
- Build mart models — final analytics-ready tables for the API layer
- Write dbt tests: uniqueness, not-null, accepted values, referential integrity
- Write dbt documentation and descriptions for all models and columns
- Configure dbt snapshots for slowly changing dimension tracking

■ Estimated Effort: **1–1.5 weeks**

OUTPUT

A fully tested, documented dbt transformation layer producing analytics-ready models.

■ Pipeline Automation, Analytics & Quality (Phases 6 – 8)

06

Airflow Orchestration & Automation

Data Engineering



Goal: Automate the entire pipeline to run on a schedule with zero manual intervention.

■ Automation Highlight: Every 6 hours Airflow triggers → fetches new jobs → cleans & deduplicates → dbt refreshes models → PostgreSQL is updated → API serves fresh data → App shows latest insights.

Tasks:

- Design the DAG structure (one DAG per source or a single master DAG)
- Write Airflow DAGs that trigger ingestion → cleaning → dbt runs in sequence
- Set schedule intervals (e.g. every 6 hours or daily)
- Add retry logic, failure alerts (email/Slack), and detailed logging
- Implement incremental-run checkpointing so each run only processes new data
- Test the full end-to-end automated pipeline run from source to analytics
- Monitor DAG health via the Airflow UI and set SLA alerts

■ Estimated Effort: 1 week

OUTPUT

A fully automated pipeline that continuously keeps the database updated without manual runs.

07

Analytics & Insights Models

Data Engineering



Goal: Build the analytical SQL models that answer real Egyptian labor market questions.

Tasks:

- Most in-demand job roles right now (by posting volume)
- Most oversupplied job roles (high supply vs demand ratio)
- Salary ranges by job title, city, seniority level, and industry
- Top hiring companies: volume, sector, and location breakdown
- Most requested skills and technologies per role
- Job distribution by city and governorate across Egypt
- Hiring trends over time: weekly and monthly patterns
- Remote vs on-site vs hybrid distribution analysis

■ Estimated Effort: 1 week

OUTPUT

A complete set of queryable analytical views in PostgreSQL ready to be served by the backend API.

08

Testing, Monitoring & Documentation

Data Engineering

■

Goal: *Make the pipeline reliable, observable, and understandable for the team and users.*

Tasks:

- Write data quality tests with dbt tests and custom SQL checks
- Monitor pipeline runs via Airflow UI and set up failure dashboards
- Set up alerting for pipeline failures and data quality violations
- Document the full pipeline: data sources, schema, and transformation logic
- Write a comprehensive README for the project repository
- Create a data dictionary for all tables, columns, and business definitions
- Conduct a final end-to-end pipeline test with fresh data

■ Estimated Effort: 4–5 days

OUTPUT

A production-grade, well-tested, fully documented, and monitored data pipeline.

■ Software Engineering Phases (Phases 9 – 11)

09

Backend API Development

Software
Engineering



Goal: *Build a fast, reliable API layer that exposes analytics data to the frontend.*

Tasks:

- Set up FastAPI project and connect to PostgreSQL
- Build endpoints for salary data, job demand, company insights
- Build search endpoints: by job title, city, skill, company
- Implement pagination, filtering, and sorting on all endpoints
- Add caching layer (Redis) for frequently requested analytics
- Secure the API with rate limiting and input validation
- Write API documentation with Swagger/OpenAPI

■ Estimated Effort: **1–1.5 weeks**

OUTPUT

A production-ready FastAPI backend serving all analytics to the frontend.

10

Frontend Web Application

Software
Engineering



Goal: *Build the user-facing web app that makes Egypt's job market data accessible to everyone.*

Tasks:

- Set up React / Next.js project with Tailwind CSS
- Build the home dashboard: market overview and key metrics
- Build salary explorer: filter by title, city, seniority
- Build job demand page: in-demand vs oversupplied roles chart
- Build company profiles: hiring activity, roles, and locations
- Build skills trends page: top skills per role/industry
- Build geographic map: job distribution across Egyptian cities
- Implement job search with filters (city, salary, type, skill)

■ Estimated Effort: **2–3 weeks**

OUTPUT

A polished, publicly accessible web application displaying Egypt's job market insights.



Goal: *Deploy the full system to the cloud so it runs reliably and is publicly accessible.*

Tasks:

- Dockerize all components: Airflow, dbt, FastAPI, PostgreSQL
- Write docker-compose for local orchestration
- Deploy PostgreSQL to a managed cloud database (e.g. Supabase / RDS)
- Deploy FastAPI backend to a cloud server (e.g. Railway / Render / EC2)
- Deploy frontend to Vercel or Netlify
- Set up CI/CD pipeline with GitHub Actions
- Configure environment variables, secrets management, and monitoring

■ Estimated Effort: **1 week**

OUTPUT

The full Job Market Tracker platform live on the internet, automatically updating.

Full Project Summary

#	Phase Name	Domain	Effort
0			
1	Environment Setup & Project Structure	■ ■ Data Engineering	2–3 days
0			
2	Data Source Research & Ingestion Scripts	■ ■ Data Engineering	1.5–2 wks
0			
3	Data Cleaning & Standardization	■ ■ Data Engineering	1–1.5 wks
0			
4	Data Modeling & PostgreSQL Schema Design	■ ■ Data Engineering	4–5 days
0			
5	dbt Transformation Layer	■ ■ Data Engineering	1–1.5 wks
0			
6	Airflow Orchestration & Automation	■ ■ Data Engineering	1 week
0			
7	Analytics & Insights Models	■ ■ Data Engineering	1 week
0			
8	Testing, Monitoring & Documentation	■ ■ Data Engineering	4–5 days
0			
9	Backend API Development	■ Software Engineering	1–1.5 wks
1			
0	Frontend Web Application	■ Software Engineering	2–3 wks
1			
1	Deployment & Infrastructure	■ Software Engineering	1 week

■ ■ Data Engineering Total

7 – 9 weeks

■ Software Engineering
Total

4 – 5.5 weeks

■ Full Project Total

11 – 14.5 weeks

Technology Stack

Layer	Technology	Purpose
-------	------------	---------

Ingestion	Python (requests, BeautifulSoup, feedparser)	Collect raw data from APIs, RSS, scraping
Orchestration	Apache Airflow	Schedule & automate pipeline runs
Transformation	dbt (data build tool)	SQL-based modeling & testing
Storage	PostgreSQL	Relational database for all data
Backend API	FastAPI (Python)	Serve analytics data to the frontend
Frontend	React / Next.js + Tailwind CSS	User-facing web application
Deployment	Docker + GitHub Actions + Cloud	Containerization, CI/CD, hosting
Version Control	Git + GitHub	Code management and collaboration